

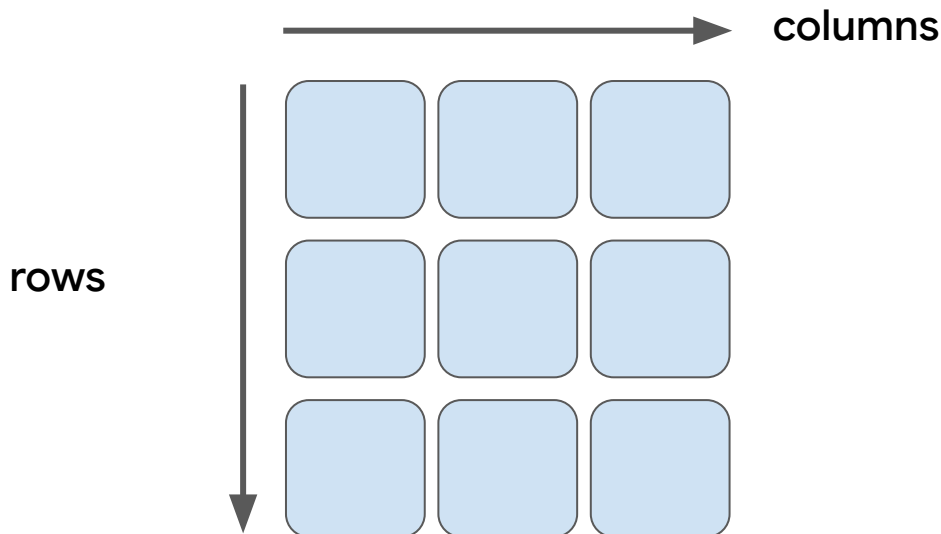
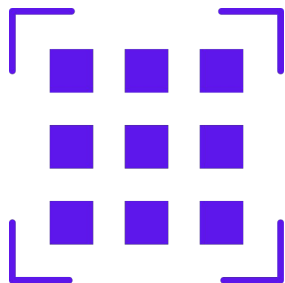
Matrices



Matrices



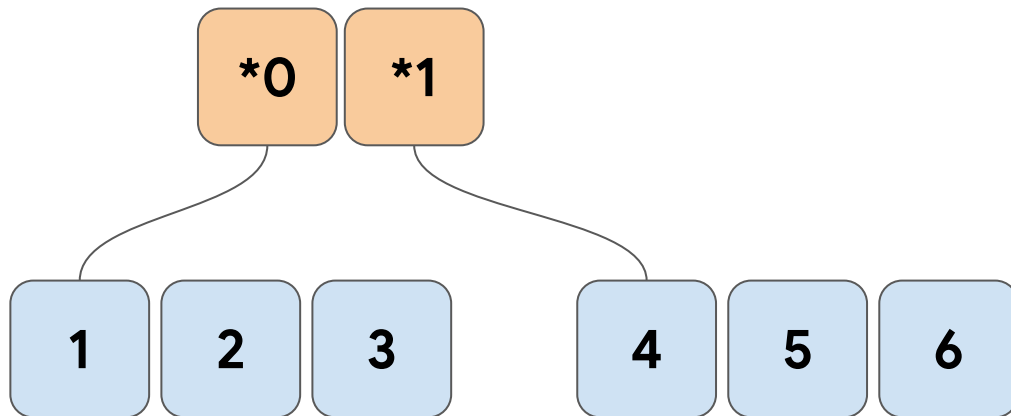
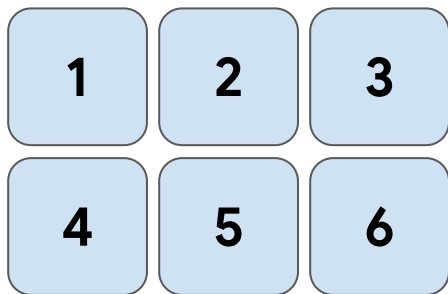
- Two-dimensional arrays can be defined as **arrays within an array**.
- **2D** arrays erected as matrices, which is a collection of rows and columns.



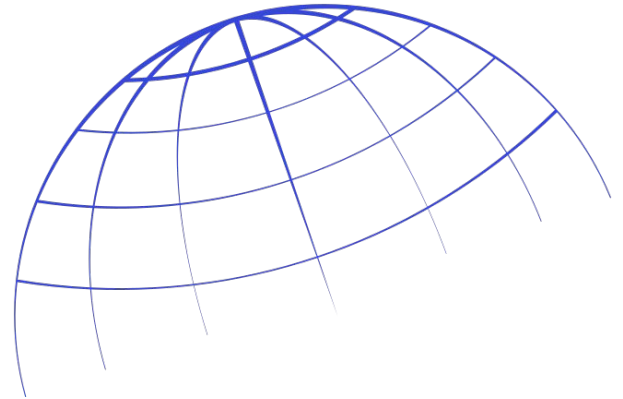
Matrices



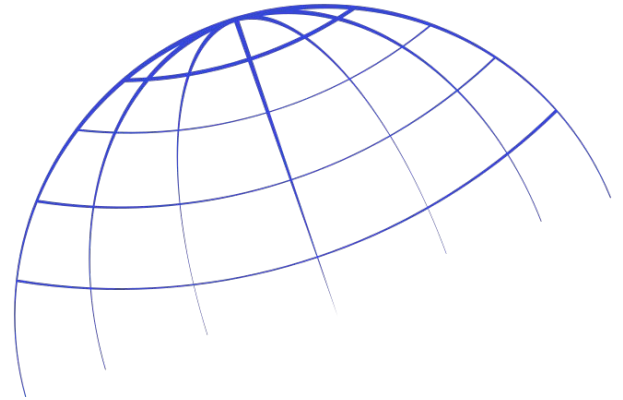
- You should think two dimensional arrays as **matrices**.
- In reality, they are arrays holding references to other arrays



Common Approaches



Traversing Matrices



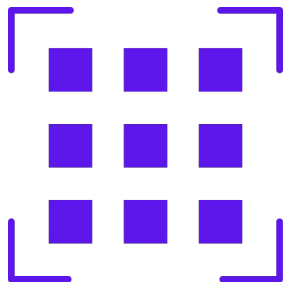
Traversing - from Top-left to Bottom-right

Approach

- Start from (0, 0)
- Increment column index by one for each row you visit

Implementation

```
for row_idx in range(len(matrix)):
    for col_idx in range(len(matrix[0])):
        print(matrix[row_idx][col_idx])
```



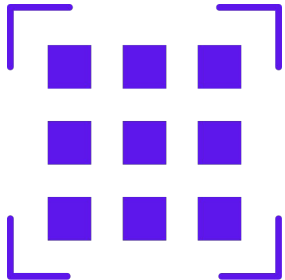
Traversing - From Bottom-right to Top-left

Approach

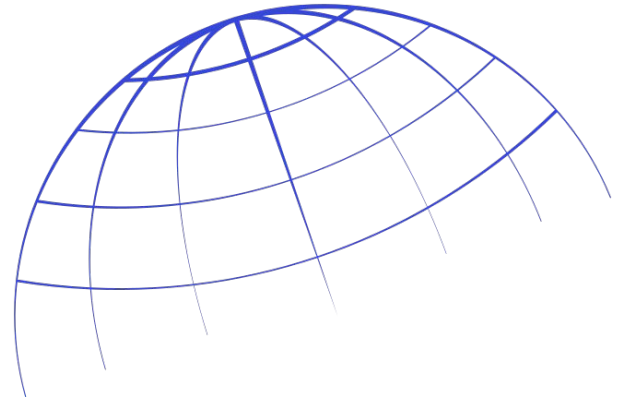
- Start from (last_row - 1, last_col - 1)
- Decrement column index by one for each row you visit.

Implementation

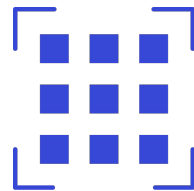
```
for row_idx in range(len(matrix) - 1, -1, -1):  
    for col_idx in range(len(matrix[0]) - 1, -1, -1):  
        print(matrix[row_idx][col_idx])
```



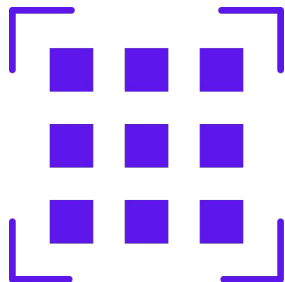
Enumerating Cells/ 2D - 1D Mapping



Enumerating Cells - Approach



	0	1	2	3
0	0	1	2	3
1	4	5	6	7
2	8	9	10	11

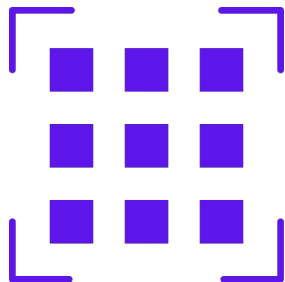


Enumerating Cells - Approach

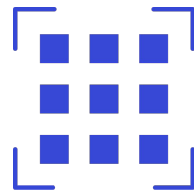


Consecutive elements in a column differ by **the number of columns**

	0	1	2	3
0	0	1	2	3
1	4	5	6	7
2	8	9	10	11

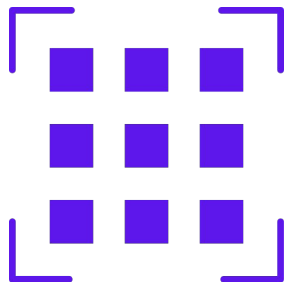


Enumerating Cells - Approach

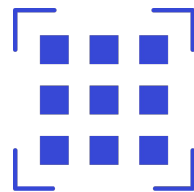


Consecutive elements in a row differ by 1

	0	1	2	3
0	0	1	2	3
1	4	5	6	7
2	8	9	10	11

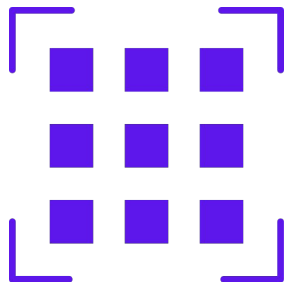


Enumerating Cells - Approach



Therefore the 1D number of a cell is $\text{row_number} * \text{n_columns} + \text{column_number}$

	0	1	2	3
0	0	1	2	3
1	4	5	6	7
2	8	9	10	11

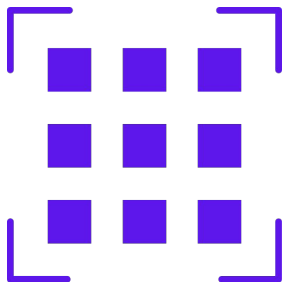


Enumerating Cells - Remainder Theorem

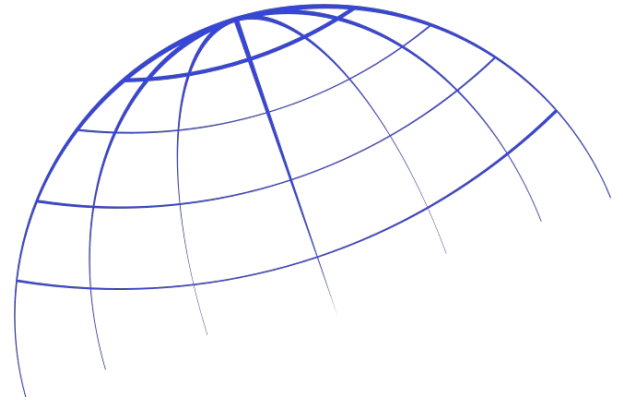
```
cell_number = row_number*n_columns + column_number
```

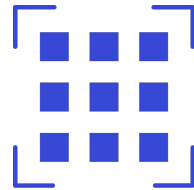
```
row_number = cell_number // n_columns
```

```
column_number = cell_number % n_columns
```



Diagonal Keys

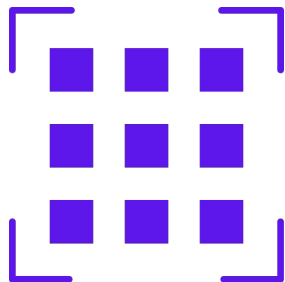


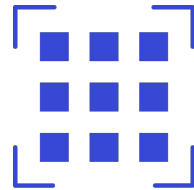


Diagonal Keys

What is common about the highlighted column and row numbers?

	0	1	2	3
0	0	1	2	3
1	4	5	6	7
2	8	9	10	11

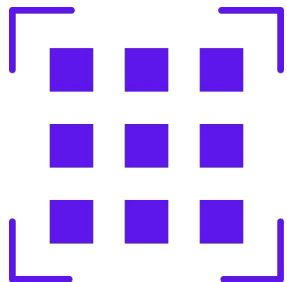




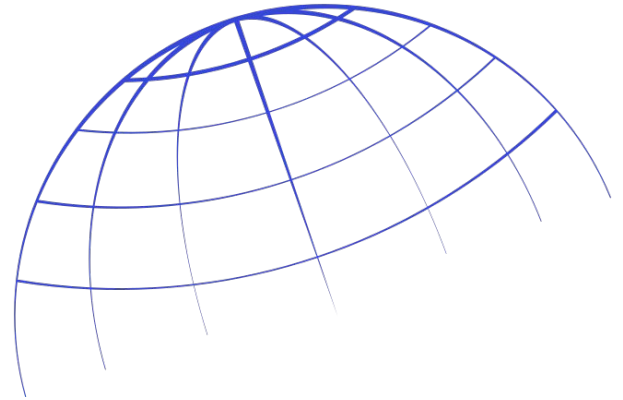
Diagonal Keys

What is common about the highlighted column and row numbers?

	0	1	2	3
0	0	1	2	3
1	4	5	6	7
2	8	9	10	11



Common Pitfalls



Common Pitfalls - Index Out of Bound

- Trying to access elements using indices that are **greater (or equal to)** than the size of our array will result in this exception.

Testcase Result 

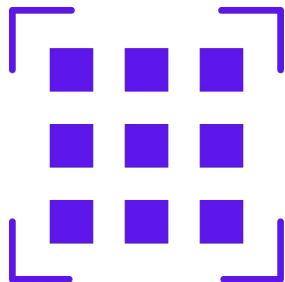
```
IndexError: list index out of range
  print(nums[i])
Line 4 in twoSum (Solution.py)
    ret = Solution().twoSum(param_1, param_2)
Line 28 in _driver (Solution.py)
    _driver()
Line 39 in <module> (Solution.py)
```

Stdout

2

Console ^

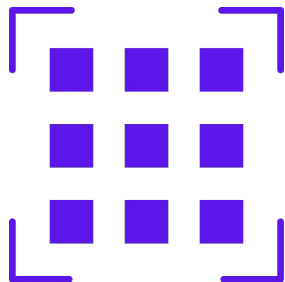
 Run Submit



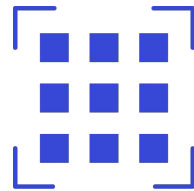
Common Pitfalls - Negative Indexing

- Because we may use negative indices in Python, it is a typical problem to use negative indices accidentally, which can result in a highly troublesome misunderstanding if no exception is triggered.

Negative indices:	-4	-3	-2	-1
Positive indices:	0	1	2	3
Values:	[1	, 2	, 3	, 4]

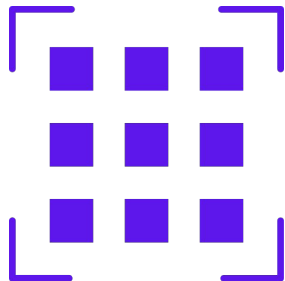


Common Pitfalls - Copying lists



- A list is a reference. Take care when you want to you want to pass **only the content** of the list.

```
nums[::]  
nums.copy()
```

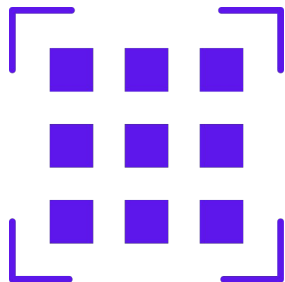


Common Pitfalls - Initializing with *

- The `*` operator can be used as `[object]*n` where `n` is the number of elements in the array.
- In the case of 2D arrays, this will result in **shallow lists**
- Hence using list comprehensions is a safer way to create 2D lists.

```
row = len(matrix)
col = len(matrix[0])
```

```
transposed = [[0]*rows for _ in range(cols)]
```



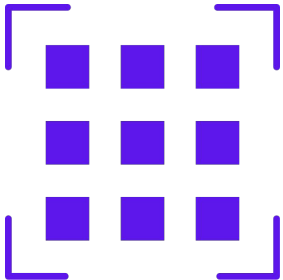
Common Pitfalls - Using insert, pop and in

- insert - inserts a given element at a given index
 - `insert(i, value)` : $O(N)$
- pop - returns the value at a given index and removes it from the list where the default is the last index
 - `pop` : $O(1)$
 - `pop(i)` : $O(n)$
- The `in` operator returns a boolean denoting whether an element is present in the iterable data structure we use it on.
 - Lists $O(N)$
 - Map, Set $O(1)$

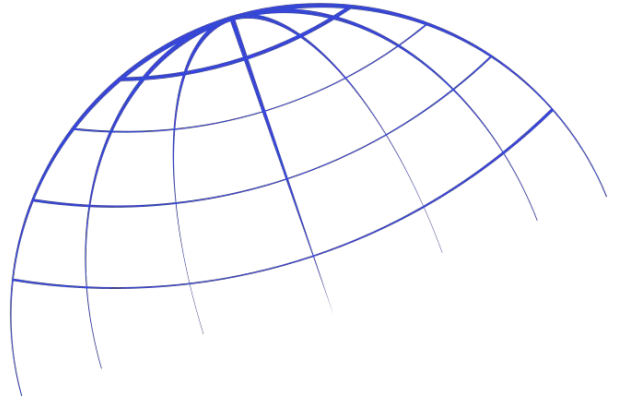
Common Pitfalls - Mistaking Row Iterators with Column iterators

- It is pretty common to mistake your `i` and `j`'s representing rows and columns. And sometimes, especially when the matrix is a square one, it does not throw an exception but hours of confusion

```
for col_idx in range(len(matrix[0])):
    for row_idx in range(len(matrix)):
        print(matrix[col_idx][row_idx])
```

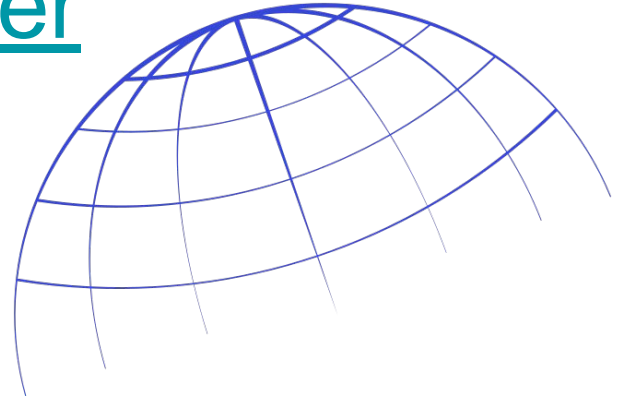


Checkpoint



Pair Programming Question

Image smoother



Practice Questions

Reverse string

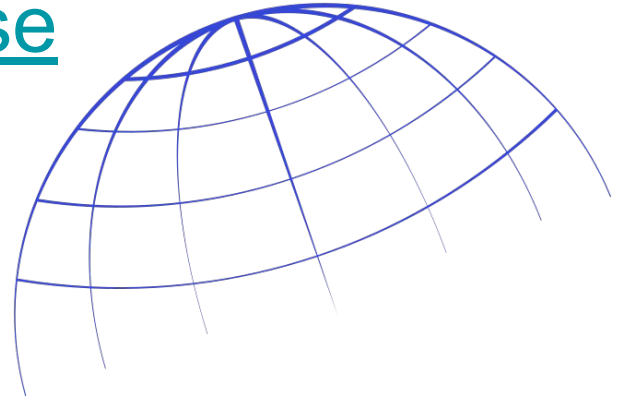
Count Equal and Divisible Pairs in an..

All Divisions With the Highest Score..

Transpose matrix

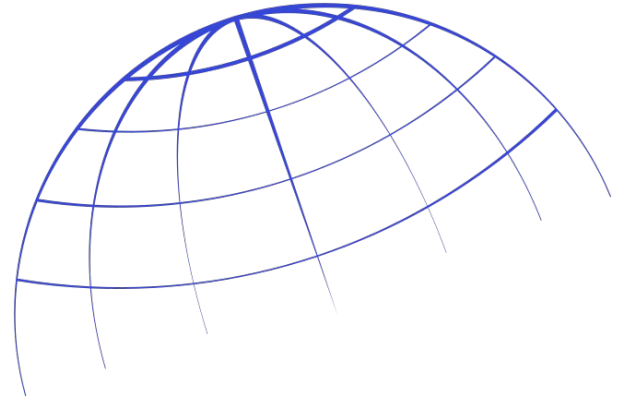
Diagonal traverse

Rotate image



Resources

- <https://neetcode.io/courses/dsa-for-beginners/0>: useful on laying out the fundamentals
- <https://www.geeksforgeeks.org/python-arrays/>: covers large range of topics for python
- [Python List pop\(\) Method - GeeksforGeeks](#): good examples
- [Python List insert\(\) Method With Examples - GeeksforGeeks](#): good examples
- [Python | Which is faster to initialize lists? - GeeksforGeeks](#): good article on list initialization



Quote of the Day

If you want to enjoy the rainbow,
be prepared to endure the storm

Warren W. Wiersbe

